School of Computing and Information Systems

COMP30023: Computer Systems

Lab Week 7

Sniffing packets with Wireshark

1 Introduction

Debugging network problems often requires the inspection of the packets being sent. Is the program sending the correct packets? Is DNS working? Can the network find which node has which IP address?

A program that captures all packets, including those not addressed to it, is said to "sniff" the network. One of the most popular network sniffers is Wireshark `https://www.wireshark.org`. This not only captures packets, but also decodes the protocol layers making which assists in understanding what is happening on the network.

In this lab, you will experiment with Wireshark. This may be useful for your project, if you develop that on your own computer.

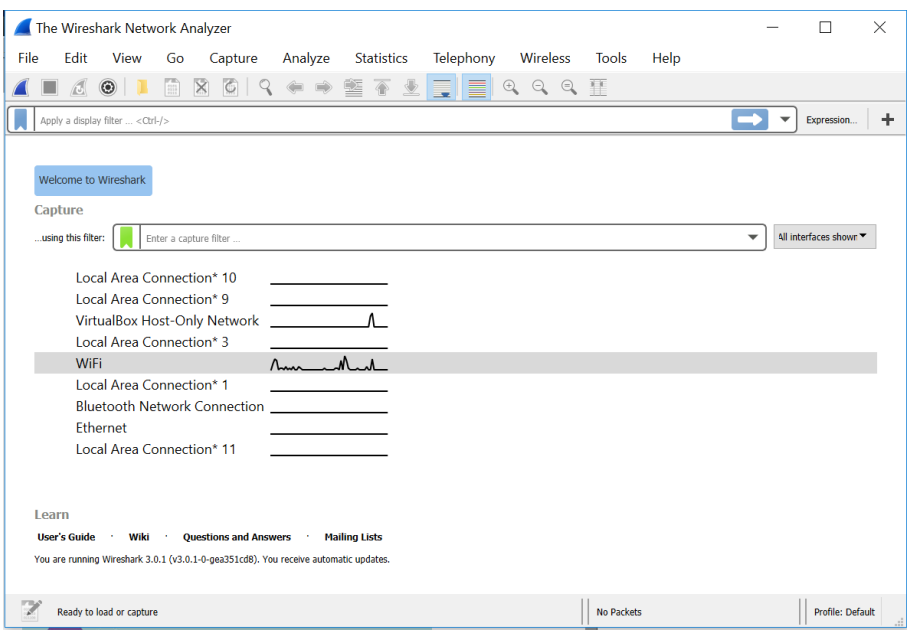
You will also practice using bit-manipulation operations in C, which will also be useful for your project, wherever you develop it.

2 Starting Wireshark

Unlike most labs, this lab should be performed on your own computer, not on your VM.

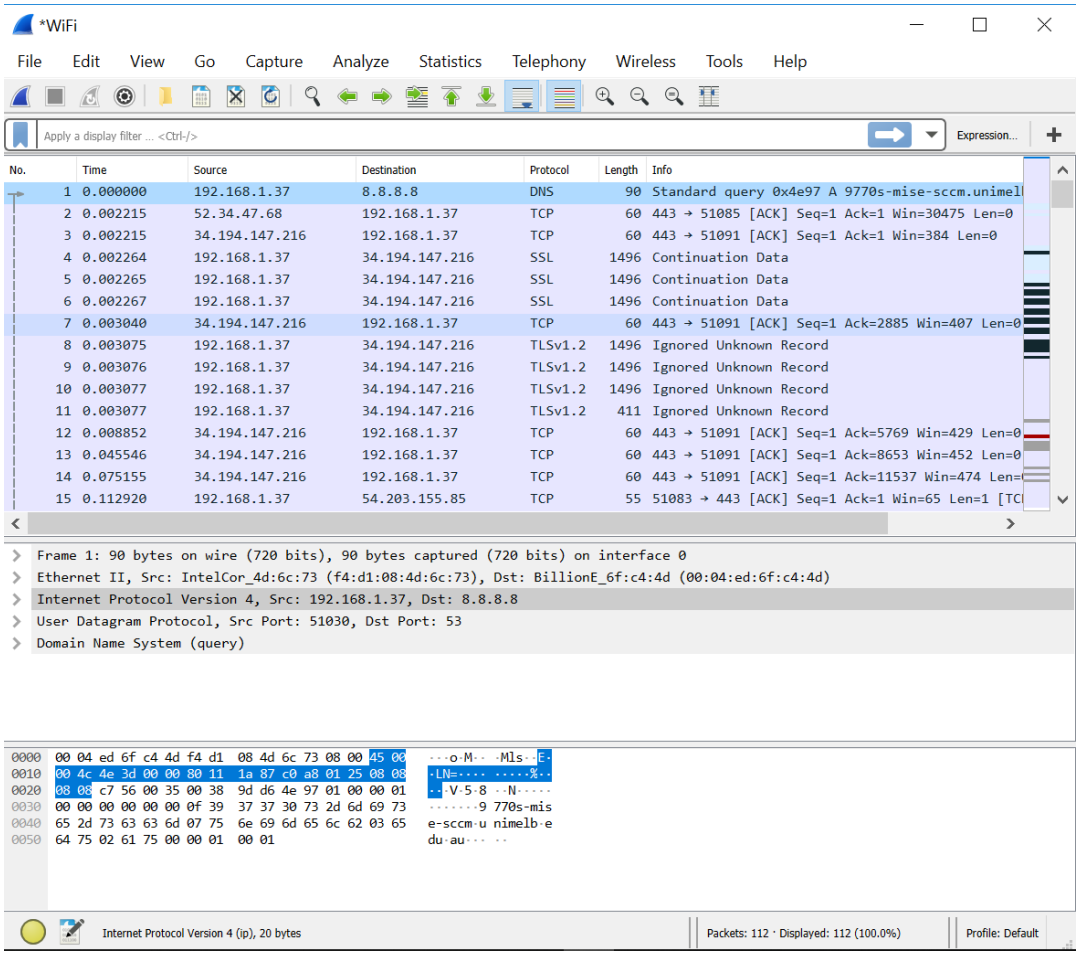
Install Wireshark from `https://www.wireshark.org`. If you are asked to install NPCAP, do not check the "Restrict NPCAP driver's access to Administrators only". If you are curious, check the "Support raw 802.11 traffic" option.

Run Wireshark. If you run Linux, run it as root. If you get an error about Lua being disabled, ignore it.



Select an interface to monitor. Wireshark shows the amount of traffic on each of the interfaces. Choose any that has traffic, except any labelled Loopback or the like.

Double click on the name of the interface. This will start filling the top pane with lines of text on coloured backgrounds. Each of these is the summary of a packet, and the background colour indicates the protocol.



Wait a minute or so to collect a good sample of traffic, and then click the red square second from the left of the toolbar (stop).

Click on the text box just under the tool bar with the grey text "Apply a display filter". Enter "tcp" and click on the icon at the end of the row with a white arrow pointing right on a greyish blue background. This will display only TCP packets. However, not all packets may display as TCP in the "protocol" field (between "destination" and "length").

Question: Why not?

Click on one of the lines representing packets. Observe the middle and bottom panels change. Click on the arrow (>) to the left of "Transmission Control Protocol" and scroll to see all of the TCP header fields.

Click on some of the fields, and notice that some of the hexadecimal numbers in the bottom pane are highlighted.

Question: Can you relate these numbers to the field that you clicked on? How does this relate to the concepts of *encapsulation* and layers?

Question: How many bytes per packet seem to be overhead? How many seem to be application layer payload (i.e., user data, without all of the headers)?

Identify as many protocols as you can from the Wireshark output. You don't need to work out what each one does yet; just find their names. Clear the filter text box to look at protocols other than TCP. Try doing different network-based activities: web browsing, Zoom, sending and

receiving email (if you have a non-web mail client), streaming audio or video, using ssh to your VM.
Question: Based on where in the packet their headers are, can you identify which protocols are at higher levels and which are at lower levels?
You may choose to have a challenge with your friends or classmates: each time you find a protocol that nobody has listed yet, post it to a chat service. (Please leave Zoom chat free for those wanting to ask questions.) See who gets the most "firsts" at the end of class.
Once you have collected several protocols, try to work out the levels they operate at. (Wikipedia can help with most of them.)
Question: How many protocols did you see at each level? Would that be different if you were at a different point in the network?

3 Domain Name System (DNS)

The domain name system is a distributed database that, among other tings, translates human-readable (text) names for hosts, like `www.unimelb.edu.au` into machine-readable (numeric) IP addresses used to direct packets to the hosts.
Replace the contents of the filter text box below the toolbar by "dns". Click on "Domain Name System" in the middle pane and scroll to the bottom. Observe the hyperlink in blue showing the matching request or response. The protocol analyser performs this matching to help diagnose problems where the DNS server does not respond, or responds very slowly.
Question: What DNS packets do you see if you repeatedly access the same host? What happens when you access a new one?

4 Bit and byte operations

The output from Wireshark has shown that many fields in network headers are smaller than a byte, or multiple bytes. We will now explore how to extract the integer values from these fields.

4.1 Combining bytes

As you know, packets and TCP streams represent a sequence of bytes. Integers over 255 must be represented by multiple bytes (as 255 the largest unsigned number representable by 8 bits). The question is, which byte should be first in the sequence? "Network" byte ordering is that the most significant byte is always first and the least significant byte is last. The first byte is stored at a lower memory address within a buffer, and so these numbers are in "big endian" order. Most – but not all – computers currently use "little endian" order (although this is just a fashion; big endian has been popular in the past and may be again). That means that we cannot simply read the data in the buffer "as if" it were a `short int`, `int` or `long long int`.
The C library provides functions that will portably convert the network order bytes into a short or long integer, and the reverse. These functions are

ntohs	network to host, short (16 bits)
ntohl	network to host, long (32 bits)
htons	host to network, short (16 bits)
htonl	host to network, long (32 bits)

Run the following code and explain its output.

```
#include <stdio.h>
#include <arpa/inet.h>
int main () {
    char a[4];
    int *b = (void*)a;
    short int *c = (void*)a;

    *b = htonl (1);
    printf ("*b %d  *c %d\n", *b, *c);

    *b = htonl (65536);
    printf ("*b %d  *c %d\n", *b, *c);
}
```

Try adding `*c = htons(1);` at different places in the code and see what happens.
Question: Can you explain it?
Question: Can you use this approach to determine if the machine you are running on is big-endian or little-endian?

4.2 Bit operations

Consider the following extract from bytes 12 and 13 of a TCP header:

```
                                1  1  1  1  1  1
 0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
 -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -
|           |       | N  C  E  U  A  P  R  S  F
|data offset|reservd| S  W  C  R  C  S  S  Y  I
|           |       |    R  R  G  K  H  T  N  N
```

How can we read the ACK (acknowledgement) flag from bit 11, or the data offset field from bits 0 to 3?
C was designed for this sort of bit manipulation, and has several useful operators:

`&`	bitwise AND	(1 `&` 0 = 0)
`|`	bitwise OR	(1 `|` 0 = 1)
`^`	bitwise XOR	(1 `^` 0 = 1, 1 `^` 1 = 0)
`>>`	shift right	(4 `>>` 1 = 2)
`<<`	shift left	(4 `<<` 1 = 8)

With these, we can read `ACK = (buf[13] >> 4) & 1`.
Question: How else could you do this?

You can read the data offset using a single bit manipulation operation.
Question: How?

You can read the reserved field using two operators.
Question: How could you do that?

There is a very important difference between the types `char` (a signed value) and `unsigned char`.
Question: What is `((unsigned char)200) >> 1`? What is `((char)(-56)) >> 1`? What is `((char)100 + (char)100) >> 1`? Explain why these are sensible answers. Should you usually use signed or unsigned values for manipulating bit fields?

A Sample solutions

A.2 Starting Wireshark

Some TCP packets will have a higher level protocol decoded. Wireshark will show the highest protocol it can decode for each packet.

The highlighted bytes are the bytes in the header that contain the selected field. Each layer will add a header (and sometimes a footer) to the packets that it carries for lower layer protocols.

It varies depending on what protocols are carried in the packet, and also the options used. Just count the header bytes for a given packet.

Higher layer protocols have their headers deeper (later) in the packet.

The protocols seen at different points of the network would be different. If you have a wireless connection, you will have a different layer 1 and 2 protocol than if you on Ethernet. If you are in the core (the middle bit that carries most of the traffic), you may be on neither of those, such as SDH/SONET or MPLS.

A.3 DNS

If you repeatedly access the same host, you should see very few DNS packets (only when a table entry times out and has to be replaced). If you access a new host, you will see a DNS exchange (request / response).

A.4 Bit and Byte Operations

A.4.1 Combining bytes

1. The value 1 is stored as 0x01000000 in host byte order (assuming little-endian). When converted to network byte order, it is 0x00000001 (i.e. bytes 0 0 0 1). This value is stored to 'b'. When read, it represents 0x01000000 (decimal 16777216).

   The value 65536 is stored as 0x00000100 in host byte order (assuming little-endian). When converted to network byte order, it is 0x00010000 (i.e. bytes 0 1 0 0). This value is stored to 'b'. When read, it represents 0x00000100 (decimal 256).

   Stepping through the code:

```
char a[4];
int *b = (void*) a;         // (a[0], a[1], a[2], a[3])
short init *c = (void*) a;  // (a[0], a[1])

// htonl will be convert to network-byte order
// Hence, *b will be set to [0x00, 0x00, 0x00, 0x01]
*b = htonl(1);

// On a little-endian machine
// b will be read as 0x01000000, which is 16777216.
// c will be read as 0x0000, which is 0.
printf ("*b %d  *c %d\n", *b, *c);

// 65536 in hexadecimal is 0x00010000
// Now, *b will be set to [0x00, 0x01, 0x00, 0x00]
```

```
*b = htonl(65536);

// On a little-endian machine
// b will be read as 0x00000100, which is 256.
// c will be read as 0x0100, which is 256.
printf("*b %d  *c %d\n", *b, *c);
```

2. On big-endian machines, `htonl` does nothing.
   On little-endian machines, it reverses the order of the bytes.

A.4.2 Bit operations

1. `(buf[13] & 16) >> 4`,
   i.e. `(buf[13] & 0b1000) >> 4`

   Or Bitwise AND with comparison

2. `buf[12] >> 4`

3. `(buf[12] >> 1) & 7`, where `7` is `0b111` (rightmost 3 bits)
   Or Bitwise AND and Bitwise right shift

4. `100 -28 100`
   These are sensible when using "`>> x`" to divide by 2^x.
   You should usually use unsigned types for manipulating bit fields.